

# Implementation Guide — Agent-Generated Preference Data for DPO Fine-Tuning

Step-by-step, runnable pipeline: environment setup, agent debate, dataset assembly, DPO training, evaluation, and explainability.

---

Project: Agent-Generated Preference Data for DPO Fine-Tuning of a Small Language Model  
Prepared for: Dhruv Singhal — CSE, RV College of Engineering (RVCE), Bengaluru — USN: 1RV24CS077  
Document version: 1.0 | September 2026

**Budget constraint:** Every component of this project is designed to run at \$0 cost — free-tier compute (Google Colab T4), free/open-weight models, and free public datasets only.

## 1. Environment Setup (Google Colab, free T4)

```
# Runtime -> Change runtime type -> T4 GPU (free tier)

!pip install -q -U "unsloth[colab-new] @ git+https://github.com/unslothai/unsloth.git"
!pip install -q -U trl peft accelerate bitsandbytes datasets sentence-transformers shap scikit-learn
!pip install -q -U huggingface_hub

from huggingface_hub import login
login() # paste a free HF token (needed only to push adapters/datasets, optional)
```

## 2. Step 1 — Agent Debate & Critic Pipeline (data generation only, no training)

This layer never touches model weights. Use either a free-tier hosted API (Groq's free tier is fast and has a generous free quota) or a local model via Ollama for zero rate-limit risk. Pseudocode below is provider-agnostic — swap in whichever `call_llm()` you have free access to.

```
def generate_agent_answers(prompt, n_agents=4):
    personas = [
        "Answer directly and concisely.",
        "Answer thoroughly, showing your reasoning step by step.",
        "Answer as a domain expert, prioritizing technical precision.",
        "Answer as a careful skeptic who flags any uncertainty.",
    ][:n_agents]
    return [call_llm(system=p_, user=prompt) for p_ in personas]

def critic_rank(prompt, answers, use_devils_advocate=True):
    rubric = ("Score each answer 1-5 on helpfulness and correctness. "
              "Pick the single best and single worst answer.")
    if use_devils_advocate:
        minority = call_llm(
            system="Argue for the answer you think is currently underrated.",
            user=f"Prompt: {prompt}\nAnswers: {answers}")
        answers = answers + [minority]
    verdict = call_llm(
        system=f"You are an impartial critic. {rubric}",
        user=f"Prompt: {prompt}\nAnswers: {answers}\nReturn JSON: "
             f"{{'chosen': <idx>, 'rejected': <idx>, 'scores': {{...}}}}")
    return parse_json(verdict)

def build_preference_dataset(seed_prompts, out_path):
    with open(out_path, "w") as f:
        for prompt in seed_prompts:
            answers = generate_agent_answers(prompt)
            verdict = critic_rank(prompt, answers)
            row = {
                "prompt": prompt,
                "chosen": answers[verdict["chosen"]],
                "rejected": answers[verdict["rejected"]],
                "meta": {"source": "agent_pipeline", **verdict},
            }
            f.write(json.dumps(row) + "\n")
```

## 3. Step 2 — Assemble the Two Comparison Datasets

```

from datasets import load_dataset, Dataset
import json

# A) Agent-generated dataset (built above)
agent_ds = Dataset.from_json("agent_preferences.jsonl")

# B) Equal-sized human-labeled reference set from HH-RLHF, SAME seed prompts where possible
hh = load_dataset("Anthropic/hh-rlhf", split="train")
hh_subset = hh.select(range(len(agent_ds))) # match sample size for a fair ablation
hh_subset = hh_subset.rename_columns({"chosen": "chosen", "rejected": "rejected"})

# TRL's DPOTrainer expects columns: "prompt", "chosen", "rejected"
def to_dpo_format(example):
    return {"prompt": example["prompt"],
            "chosen": example["chosen"],
            "rejected": example["rejected"]}

agent_ds = agent_ds.map(to_dpo_format)
hh_subset = hh_subset.map(to_dpo_format)

```

## 4. Step 3 — DPO Training (Unsloth + TRL, LoRA)

```

from unsloth import FastLanguageModel, PatchDPOTrainer, is_bfloat16_supported
PatchDPOTrainer()
from trl import DPOTrainer, DPOConfig

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="Qwen/Qwen2.5-1.5B-Instruct",
    max_seq_length=1024,
    load_in_4bit=True,
)
model = FastLanguageModel.get_peft_model(
    model,
    r=16, lora_alpha=32, lora_dropout=0.05,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                    "gate_proj", "up_proj", "down_proj"],
)

def train_dpo(dataset, run_name):
    config = DPOConfig(
        output_dir=f"outputs/{run_name}",
        per_device_train_batch_size=2,
        gradient_accumulation_steps=8,
        num_train_epochs=2,
        learning_rate=1e-5,
        beta=0.1,
        optim="adamw_8bit",
        bf16=is_bfloat16_supported(),
        fp16=not is_bfloat16_supported(),
        logging_steps=5,
        save_strategy="epoch",
        seed=3407,
    )
    trainer = DPOTrainer(model=model, args=config,
                        train_dataset=dataset, tokenizer=tokenizer)
    trainer.train()
    model.save_pretrained(f"outputs/{run_name}/adapter")
    return trainer

trainer_agent = train_dpo(agent_ds, "dpo_agent_data")
# reload a fresh base+LoRA before the second run so the two are independent

```

```
trainer_human = train_dpo(hh_subset, "dpo_human_data")
```

## 5. Step 4 — Evaluation Harness

```
def generate(model, tokenizer, prompt, max_new_tokens=256):
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    out = model.generate(**inputs, max_new_tokens=max_new_tokens)
    return tokenizer.decode(out[0], skip_special_tokens=True)

def judge_win_rate(base_model, tuned_model, tokenizer, eval_prompts, judge_call):
    wins = ties = losses = 0
    for prompt in eval_prompts:
        a = generate(base_model, tokenizer, prompt)
        b = generate(tuned_model, tokenizer, prompt)
        verdict = judge_call(prompt, a, b) # returns "A", "B", or "tie"
        wins += verdict == "B"
        losses += verdict == "A"
        ties += verdict == "tie"
    n = len(eval_prompts)
    return {"win_rate": wins/n, "tie_rate": ties/n, "loss_rate": losses/n}

# Benchmark scoring: TruthfulQA / HaluEval subset
from datasets import load_dataset
tqa = load_dataset("truthful_qa", "generation", split="validation").select(range(200))
# score with your own automated grader (exact-match / judge-scored) per model
```

## 6. Step 5 — Ablation Table Assembly

Run the evaluation harness against three models: the untouched base model, the agent-data DPO model, and the human-data DPO model, on the identical held-out prompt set, and assemble a single results table:

| Model variant                      | Win-rate metric  | TruthfulQA          | HaluEval          |
|------------------------------------|------------------|---------------------|-------------------|
| Base model (no DPO)                | — (reference)    | baseline            | baseline          |
| DPO — agent-generated data         | win-rate vs base | TruthfulQA $\Delta$ | HaluEval $\Delta$ |
| DPO — human-labeled data (HH-RLHF) | win-rate vs base | TruthfulQA $\Delta$ | HaluEval $\Delta$ |

Report the agent-vs-human gap explicitly (percentage points) — this single number is the headline result of the project.

## 7. Step 6 — Reward Model + SHAP Explainability (optional but recommended)

```
from sentence_transformers import SentenceTransformer
from sklearn.linear_model import LogisticRegression
import shap, numpy as np

embedder = SentenceTransformer("all-MiniLM-L6-v2")

def featurize(prompt, response):
    emb = embedder.encode(response)
    return np.concatenate([
```

```

        emb,
        [len(response.split())],          # length
        [response.lower().count("might")  # hedging
         + response.lower().count("possibly")],
    ])

X, y = [], []
for row in agent_ds:
    X.append(featurize(row["prompt"], row["chosen"])); y.append(1)
    X.append(featurize(row["prompt"], row["rejected"])); y.append(0)

reward_model = LogisticRegression(max_iter=1000).fit(X, y)

explainer = shap.LinearExplainer(reward_model, X)
shap_values = explainer.shap_values(X[:50])
shap.summary_plot(shap_values, X[:50]) # -> which features drive "chosen" predictions

```

## 8. Suggested Build Order (checklist)

---

- 1 Load Qwen2.5-1.5B-Instruct in Colab, confirm 4-bit load + generation works on the free T4.
- 2 Build and smoke-test `call_llm()` against your chosen free API or local Ollama model.
- 3 Generate a small pilot batch (~20 prompts) through the agent debate + critic pipeline; manually spot-check quality before scaling up.
- 4 Scale to the full preference dataset (target: 300-800 pairs — enough for a meaningful DPO run on a 1.5B model without overfitting).
- 5 Pull an equal-sized HH-RLHF human-labeled subset.
- 6 Run DPO training on both datasets with identical hyperparameters.
- 7 Run the evaluation harness on all three model variants; assemble the ablation table.
- 8 Train the auxiliary reward model and generate the SHAP summary plot.
- 9 Write up results: methodology, ablation table, SHAP figure, discussion of what the agent critic appears to be rewarding vs. what a human rubric rewards.

## 9. Common Failure Modes to Watch For

---

- **Reward margin collapses to ~0 quickly:** learning rate too high or beta too low — drop learning rate by 5-10x or raise beta.
- **Model output degenerates (repetition, gibberish) after DPO:** too many epochs on a small dataset — reduce to 1 epoch or add more preference pairs.
- **Agent critic always prefers the longest answer:** a real and worth-reporting finding, not a bug — add response length as an explicit feature in the reward-model analysis rather than silently correcting it.
- **Colab session disconnects mid-run:** checkpoint LoRA adapters every epoch to Google Drive or the Hugging Face Hub so a run can resume.

## References & Links

---

Hugging Face. **TRL — DPOTrainer Documentation**. [https://huggingface.co/docs/trl/dpo\\_trainer](https://huggingface.co/docs/trl/dpo_trainer) — *reference implementation used for training*

Unsloth AI. **Unsloth** — **GitHub Repository & Docs**. <https://github.com/unslothai/unsloth> — *free-tier-optimized LoRA/QLoRA/DPO training*

Unsloth AI. **Unsloth Documentation**. <https://docs.unsloth.ai> — *setup and notebook guides*

Hugging Face. **PEFT Documentation**. <https://huggingface.co/docs/peft/index> — *LoRA adapter library*

Qwen Team, Alibaba. **Qwen2.5-1.5B-Instruct** — **Model Card**. <https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct> — *primary recommended base model*

Anthropic. **hh-rlhf Dataset (Hugging Face Datasets)**. <https://huggingface.co/datasets/Anthropic/hh-rlhf> — *human-labeled preference data used as reference set*

Hugging Face Datasets. **truthful\_qa Dataset**. [https://huggingface.co/datasets/truthful\\_qa](https://huggingface.co/datasets/truthful_qa) — *evaluation dataset*

RUCAIBox. **HaluEval** — **GitHub Repository (data + code)**. <https://github.com/RUCAIBox/HaluEval> — *evaluation dataset and scoring scripts*

Reimers, N., Gurevych, I. / SBERT.net. **Sentence-Transformers Documentation**. <https://www.sbert.net> — *embedding model used for reward-model features*

Lundberg, S. et al. **SHAP** — **GitHub Repository**. <https://github.com/shap/shap> — *explainability library used for reward-model analysis*

Google. **Google Colaboratory** — **Free Tier**. <https://colab.research.google.com> — *free T4 GPU compute used for all training*

Groq. **GroqCloud** — **Free API Tier**. <https://console.groq.com> — *one option for the free-tier agent-debate LLM calls*

Ollama. **Ollama** — **Run LLMs Locally**. <https://ollama.com> — *zero-cost, rate-limit-free local alternative for the agent layer*

---

*End of document.*